

Jetson 端工业缺陷检测的输入数据路径重构

王凯伦, 王琦

合肥工业大学数学学院, 安徽 合肥 230601

通信作者: 王琦

摘要: 针对 Jetson 端工业缺陷检测中网络低精度化不能直接消除输入形成与主机—设备数据移动开销的问题, 本文在固定 YOLOv8n 和 TensorRT INT8 混合精度 Engine 的条件下, 将 CPU/OpenCV 侧形成 FP32 NCHW 张量并复制至设备的基线路径, 重构为 packed BGR 原始图像暂存、二维主机到设备复制以及 GPU 侧融合 CUDA 预处理直接形成 TensorRT 设备输入的路径。在统一的任务正确性、逐帧 E2E 延迟、进程级 FPS 和合并样本 P95/P99 评价口径下, 实验通过 V0→V2R 与 V2R→V3R 两级受控比较, 分别考察完整输入形成路径重构和 pageable→pinned 主机暂存变化。三条路径在冻结的 180 幅测试集上获得一致的任务级指标。V2R 相对 V0 达到 2.24× FPS, 平均延迟降低 55.45%, 构成主要 E2E 性能增量; V3R 相对 V2R 的 FPS 变化为 +4.07%, 平均延迟变化为 -4.03%, P95 变化为 +0.15%、P99 变化为 -0.12%, 两个尾延迟指标变化均低于 0.2%且方向相反。结果表明, 在本文固定 Jetson 平台、实现和离线回放工作负载下, 完整输入形成与数据移动路径重构带来主要平均性能收益, pinned 暂存只提供有限的平均增量, 尚无一致的尾延迟改善证据。

关键词: Jetson; 工业缺陷检测; INT8 混合精度推理; CUDA 预处理; 主机—设备数据路径

Input Data-Path Reconstruction for Industrial Defect Detection on Jetson

WANG Kailun, WANG Qi

School of Mathematics, Hefei University of Technology, Hefei 230601, China

Corresponding author: WANG Qi

Abstract: For Jetson-based industrial defect detection, reducing network precision does not by itself remove the costs of input formation and host-device data movement. With a fixed YOLOv8n detector and TensorRT INT8 mixed-precision Engine, this study restructures a baseline path that forms an FP32 NCHW tensor through CPU/OpenCV and copies it to the device into a path based on packed BGR raw-image staging, two-dimensional host-to-device copying, and fused CUDA preprocessing that directly forms the TensorRT device input. A unified protocol evaluates task correctness, source-to-pre-sink per-frame latency, process-level FPS, and pooled P95/P99 through two controlled comparisons: V0→V2R for the complete input-formation restructuring and V2R→V3R for pageable-to-pinned host staging. All three paths produce identical task-level metrics on the frozen 180-image test set. Relative to V0, V2R achieves 2.24× FPS and 55.45% lower mean latency, representing the principal end-to-end performance gain. Relative to V2R, the V3R changes are +4.07% in FPS and -4.03% in mean latency; the P95 and P99 changes are +0.15% and -0.12%, respectively, with both tail changes below 0.2% and in opposite directions. Under the fixed Jetson platform, implementation, and offline replay workload evaluated here, the complete input-formation and data-movement restructuring provides the main average-performance gain, whereas pinned staging adds only a limited mean improvement and does not provide evidence of consistent tail-latency improvement.

Keywords: Jetson; industrial defect detection; INT8 mixed-precision inference; CUDA preprocessing; host-device data path

中图分类号: TP391.41

0 引言

金属表面缺陷自动检测是工业视觉的重要应用, NEU-DET 提供开裂、夹杂、斑块、点蚀表面、轧制氧化皮和划痕等典型缺陷数据^[1,2]。现有研究通过轻量化骨干、注意力和多尺度特征融合改善检测精度与复杂度权衡^[3-5], YOLOv8 提供多尺度检测模型及训练、推理和导出能力^[6]。面向资源受限边缘平台时, 完整检测还包括预处理、推理、后处理和数据管理^[7,8], 并需在现场附近完成完整处理流程^[9]。

INT8 部署可降低网络侧计算开销, 但量化扰动仍需任务正确性约束^[10,11], 且网络低精度化不会自动消除图像解码后的主机侧输入形成和主机—设备数据移动。本文以 TensorRT 10.3 校准式 INT8 混合精度推理为固定前提; 该版本传统隐式 INT8 量化和 calibrator 接口已标记为弃用, 结论因而限定于本文软件栈^[12]。研究范围收敛到固定检测器与 Engine 条件下的输入形成位置、主机数据表示、输入复制载荷和 pageable/pinned 暂存。

王凯伦 (1999—), 男, 山东潍坊人, 工学学士, 硕士研究生, 主要研究方向为端侧人工智能推理部署与优化。

本文设置三条受控路径：V0 在主机侧形成 FP32 NCHW 张量并复制至设备；V2R 改为 packed BGR 原始图像暂存 cudaMemcpy2DAsync 二维 H2D 复制和 GPU 融合预处理，直接形成 TensorRT 设备输入；V3R 保持 CUDA 预处理、CUDA stream 和下游拓扑不变，仅将 pageable 暂存替换为 pinned 暂存。

本文的主要贡献包括两点：1) 在固定 YOLOv8n 和 TensorRT INT8 混合精度 Engine 条

件下，将主机 FP32 张量输入路径重构为原始图像 H2D 与 GPU 融合预处理，并通过 pageable/pinned 配置隔离主机暂存类型；2) 在统一的任务正确性、E2E 延迟、进程级 FPS 和合并样本 P95/P99 口径下，通过 V0→V2R 与 V2R→V3R 两级受控比较，区分完整输入路径重构的主要收益与 pinned 暂存的有限平均增量，并以每条路径 5 个独立进程考察运行级分布和尾延迟。受控路径见图 1。

受控输入数据路径工程总览

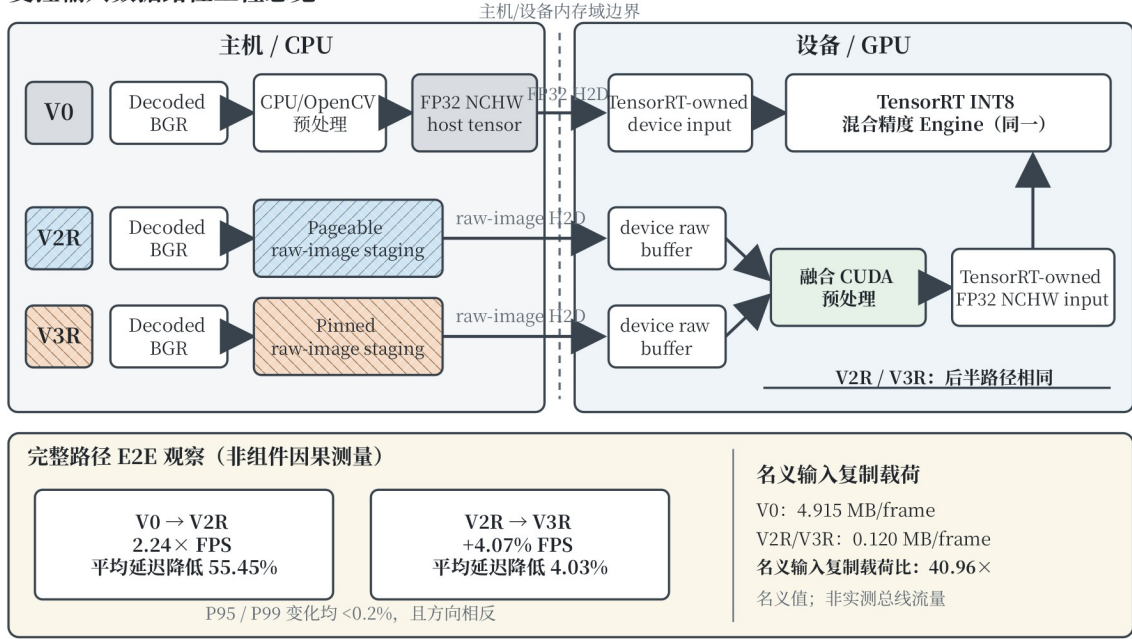


图1 V0、V2R 和 V3R 三条受控数据路径。图中数值为完整路径 E2E 观测，输入复制载荷为名义值。

1 系统对象与问题定义

1.1 模型、数据集与部署环境

本文研究部署于 NVIDIA Jetson Orin Nano Super 的 YOLOv8n 工业缺陷检测模型，输入为 640×640、batch size 为 1。数据采用 NEU-DET 目标检测标注及去重后的 split-v2，固定测试工作负载为 180 幅图像。TensorRT Engine 使用排除测试集的 1260 幅去重训练图像校准，采用 IInt8EntropyCalibrator2、batch size 1 和与生产 CPU Preprocessor 一致的 BGR letterbox、RGB/NCHW 及 FP32/255 预处理；构建启用 INT8 与 FP16 并保持 FP32 I/O，本文称其为 TensorRT INT8 混合精度 Engine。量化部署需同时检验任务性能^[13]，三条路径共用该 Engine，软件版本见表 2。

1.2 E2E 数据路径与受控变量

完整路径包括图像获取与解码、主机暂存、输入形成、必要的移动、推理与同步、后处

理和结果构造，Jetson 检测部署也通常区分预处理、推理和后处理^[14]。统一计时边界内的 E2E 耗时表示为

$$T_{E2E} = \sum_{k=1}^m T_k.$$

该式定义完整计时边界，而非仅指 TensorRT 网络推理；各阶段未独立插桩，因此不用于阶段级归因。

V0 复制主机侧 640×640 FP32 RGB/NCHW 输入；V2R/V3R 则复制 packed BGR 原始图像，并在设备侧融合完成输入形成。两条 GPU 路径使用相同 CUDA 预处理、CUDA stream 和下游拓扑，唯一差异是 pageable 或 pinned 主机暂存。锁页内存的效果取决于平台、工作负载和访问特征^[15–17]；本文依据 CUDA stream 与复制语义^[18]，在单帧顺序路径内把暂存分配类型作为独立变量。局部路径变化仍需在完整 E2E 边界评价^[19]。

1.3 统一计时边界与研究问题

平均延迟、吞吐率和尾延迟描述不同性能属性，平均值不能替代分布尾部^[20]。逐帧延迟采用 source-to-pre-sink 边界，从获取数据源帧前开始，包含图像获取与解码、路径对应的暂存和输入形成、H2D、TensorRT 推理与同步、必要的 D2H、CPU 后处理/NMS 及结果构造，在序列化与写文件前结束。进程级 FPS 以每个独立进程的 1080 个测量帧除以完整测量阶段 wall time 得到，包含统一的结果输出处理，不由逐帧延迟取倒数。

本文回答两个问题。RQ1：在固定模型、Engine 和工作负载下，将主机 FP32 张量形成路径重构为原始图像 H2D 与 GPU 融合预处理，对完整 E2E 性能产生多大影响？RQ2：在 GPU 预处理、CUDA stream 和下游拓扑保持不变时，将 pageable 原始图像暂存替换为 pinned 暂存，是否进一步改善平均性能，以及 P95/P99 是否呈现一致的尾延迟改善？

2 数据路径工程方法

三条受控路径的主要配置见表 1。

表 1 三条受控路径的特征矩阵。检测器和 TensorRT Engine 相同；三条路径均为单帧顺序执行，无跨帧流水线。

路径特征	V0	V2R	V3R
CPU 像素预处理	是	否	否
CUDA 预处理	否	是	是
主机 FP32 输入张量	是	否	否
额外打包原始图像暂存	否	Pageable	Pinned
原始图像 H2D	否	是	是
输入形成位置 / TRT 直接输入	主机 / 否	设备 / 是	设备 / 是
执行拓扑	单帧顺序	同一 TRT stream；单帧顺序	同一 TRT stream；单帧顺序

2.1 V0：主机侧 FP32 张量形成

V0 将数据源解码为主机侧 CV_8UC3 BGR 图像，CPU/OpenCV 使用 INTER_LINEAR 执行 640×640 letterbox 和常数 114 填充，再完成 BGR→RGB、HWC→CHW 与 1/255 归一化，在主机侧形成 1×3×640×640 FP32 NCHW 输入并复制至设备；Engine 推理后，输出回到主机进行检测框解码、置信度筛选和 NMS。

2.2 V2R：pageable 暂存与 GPU 输入形成

V2R 保留主机图像读取与解码，将 CV_8UC3 BGR 图像逐行复制到可复用的 std::vector<uint8_t>，形成连续 packed BGR 暂存。冻结工作负载图像为 200×200，有效行宽 600 B；letterbox 几何仍由主机按统一模型合同计算。

packed BGR 图像通过 cudaMemcpy2DAsync 写入持久化设备原始图像缓冲区，随后一个融合 CUDA 核函数完成 resize、padding、BGR→RGB、1/255 归一化和 HWC→NCHW 布局转换，直接写入 TensorRT 管理的 FP32 NCHW 设备输入。该 resize 按 V0 的 OpenCV 4.5.4 INTER_LINEAR 预处理语义建立

受控对齐合同，仅适用于本文冻结实现和工作负载，不构成通用 CUDA/OpenCV 等价性声明。CUDA 预处理器、设备缓冲区、同一 TensorRT CUDA stream 和 execution context 均跨帧复用。

2.3 V3R：pinned 原始图像暂存隔离变量

V3R 与 V2R 共享 packed BGR 语义、cudaMemcpy2DAsync、融合 CUDA 预处理、TensorRT CUDA stream、Engine 和下游拓扑，仅将主机暂存改为帧循环前由 cudaHostAlloc(..., cudaHostAllocDefault)分配的长生命周期 pinned 缓冲区。该缓冲区在全部帧之间复用，运行器结束时由 cudaFreeHost 释放；每帧仍逐行复制 width×3 字节，不逐帧分配，也不静默回退到 pageable 暂存。内存配置效果具有平台和负载依赖性^[16,17]。

三条正式路径均为单帧顺序执行，不使用 zero-copy、double buffering、multi-stream、跨帧流水线、显式传输/计算重叠或 GPU NMS；这些边界仅限定本文受测实现，两条 GPU 路径关系见图 2。

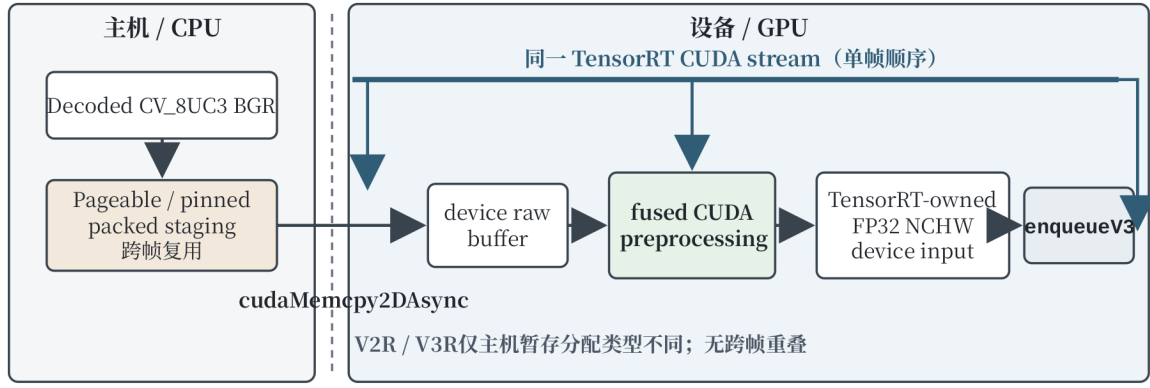


图2 V2R/V3R 主机—设备输入路径。两者仅 pageable/pinned 暂存不同；复制、融合 CUDA 预处理与 enqueueV3 沿同一 TensorRT CUDA stream 单帧顺序执行，不表示跨帧重叠。

2.4 正确性与生命周期控制

三条路径使用相同的冻结 180 幅测试图像、模型、Engine、输入尺寸、置信度阈值 0.25、IoU 阈值 0.45、max_nms=30000、max_det=300 和 class-aware 单标签后处理。V2R 在预设任务级差异门限下通过与 V0 的核验；V3R 由同一评价程序对冻结预测结果确定性重算，并以张量与检测摘要、帧顺序、处理数量、丢帧、EOS 和生命周期检查约束隔离变量身份。该正确性结论仅适用于本文冻结工作负载和评价口径。

3 实验设计

3.1 实验平台与模型配置

平台、模型和统一协议见表 2。平台与框架条件会影响部署性能^[21]，本文据此不作跨研究绝对 FPS 比较，并遵循明确数据与评价边界的基准取向^[22]。

表 2 平台、模型与统一基准协议。

项目	设置
平台	NVIDIA Jetson Orin Nano Engineering Reference Developer Kit Super
软件栈	L4T R36.5; CUDA 12.6; TensorRT 10.3; OpenCV 4.5.4
Detector / 输入	YOLOv8n; 640 × 640; batch 1
Engine	TensorRT INT8 混合精度 (INT8 + FP16 fallback); Engine 输入张量: FP32
校准	1260 张去重训练图像; IInt8EntropyCalibrator2; batch 1; 排除 test split
工作负载	固定 180 张 test 图像
路径	V0 / V2R / V3R; 单帧顺序执行
计时协议	60 帧预热; 每进程 1080 帧; 每路径 5 个独立进程
正式计时	关闭 diagnostics 与 profiling

正式运行采用 MAXN_SUPER 电源模式 (nvpmodel mode 2)，未调用 jetson_clocks，频率未独立归档。运行前温度约 46.8–47.1 °C，运行后约 48.7–49.6 °C；这些非连续观察不用于判断持续温度、运行时降频或功耗状态。

3.2 数据集与统一运行协议

性能实验使用同一 manifest 限定的 split-v2 固定 180 幅测试图像及顺序。每个独立进程预热 60 帧，同步并重置测量窗口后测量 1080 帧；每条路径执行 5 个独立进程，形成 5400 个逐帧延迟样本，三条路径共 15 个进程和 16200 个样本，不在路径间构造运行配对。

15 个独立进程按预先设定的交错顺序执行。全部进程均处理 1080 个测量帧、丢帧为 0，并通过输入顺序、EOS 和生命周期准入；逐帧 diagnostics 与 profiling 保持关闭。

3.3 正确性与性能指标

正确性采用冻结测试集和统一评价器，报告 Precision、Recall、mAP50、mAP50-95 及类别级 AP50/Recall；V3R 对冻结预测结果确定性重算并结合 V2R/V3R 摘要校验与生命周期身份检查。

对第 i 个独立进程，FPS 定义为

$$f_i = \frac{N}{T_i},$$

其中 $N=1080$ ， T_i 为测量阶段 wall time。

每条路径的平均 FPS 为 5 个独立进程级 FPS 的算术平均，误差棒为这 5 个值的样本标准差。逐帧延迟采用 source-to-pre-sink 边界，每条路径合并 5400 个样本计算均值、P95 和 P99。百分位采用 Type-7 线性插值：将延迟样本升序记为

$x_{(1)} \leq \dots \leq x_{(n)}$, 对本文 $p=0.95$ 或 $p=0.99$, 令 $h=1+(n-1)p$, $j=\lfloor h \rfloor$, $\gamma=h-j$, 则内部位置的 $Q_p=(1-\gamma)x_{(j)}+\gamma x_{(j+1)}$ 。所有统计均为描述性结果, 不进行置信区间、假设检验或统计显著性推断。

4 结果与分析

4.1 正确性

三条路径在冻结 180 幅测试图像和统一评价程序下获得相同的汇总任务指标, 类别级最大 AP50 与 Recall 路径间差异均为 0, 结果见表 3。V3R 由同一评价器对冻结预测结果确定性重算, 并非新的推理或参数选择; 该一致性不扩展为逐位相等或未来输入上的普适结论。

表 3 三条路径在冻结工作负载和统一评价程序下的任务级

正确性。	
Path	P / R / mAP50 / mAP50-95
V0	0.6913 / 0.6991 / 0.6476 / 0.3523
V2R	0.6913 / 0.6991 / 0.6476 / 0.3523
V3R	0.6913 / 0.6991 / 0.6476 / 0.3523

4.2 整体 E2E 性能

V2R 相对 V0 达到 $2.24\times$ FPS, 平均延迟降低 55.45%, 构成完整输入形成路径重构的主要 E2E 平均性能增量。V3R 相对 V2R 的 FPS 变化为 $+4.07\%$, 平均延迟变化为 -4.03% , 表明 pageable $\rightarrow$ pinned 暂存只带来有限平均增量, 整体结果见图 3。

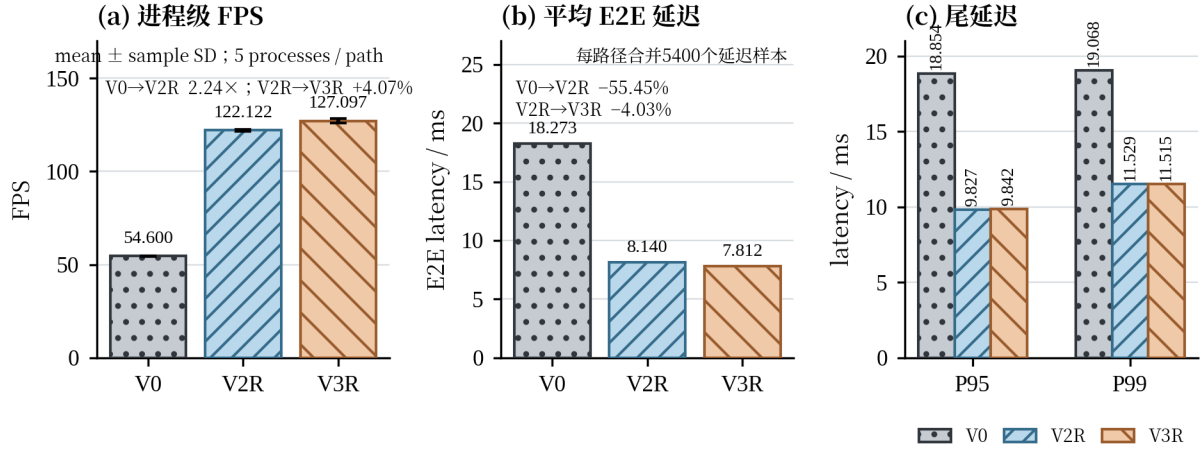


图 3 三条路径的端到端性能。(a) 为 5 个独立进程 FPS 的均值 $\pm$ 样本标准差; (b)(c) 为每条路径合并 5400 个延迟样本的均值、P95 和 P99。

4.3 数据路径分析

V0 复制 $1\times3\times640\times640\times4=4.9152$ MB/frame 的 FP32 NCHW 输入, V2R/V3R 复制 $600\times200=0.1200$ MB/frame 的 packed BGR 输入, 名义输入复制载荷比为 $40.96\times$ 。这是名义输入复制载荷, 不是实测总线流量、带宽、H2D 时间, 也不能作为测得 $2.24\times$ E2E 结果的独立因果解释。

4.4 运行级分布与尾延迟

5 个独立进程中, V2R 与 V3R 的进程级 FPS 范围分别为 121.443–122.759 和 125.595–

128.301, 进程平均延迟范围分别为 8.098–8.185 ms 和 7.740–7.894 ms, 分布见图 4。上述 FPS 范围与相应平均延迟范围均不重叠, 说明约 4% 的平均差异在 5 个独立进程的运行级分布中重复观察到, 而非仅由单一异常进程产生; 但不同路径的进程独立且不配对, 本文不据此进行显著性推断。

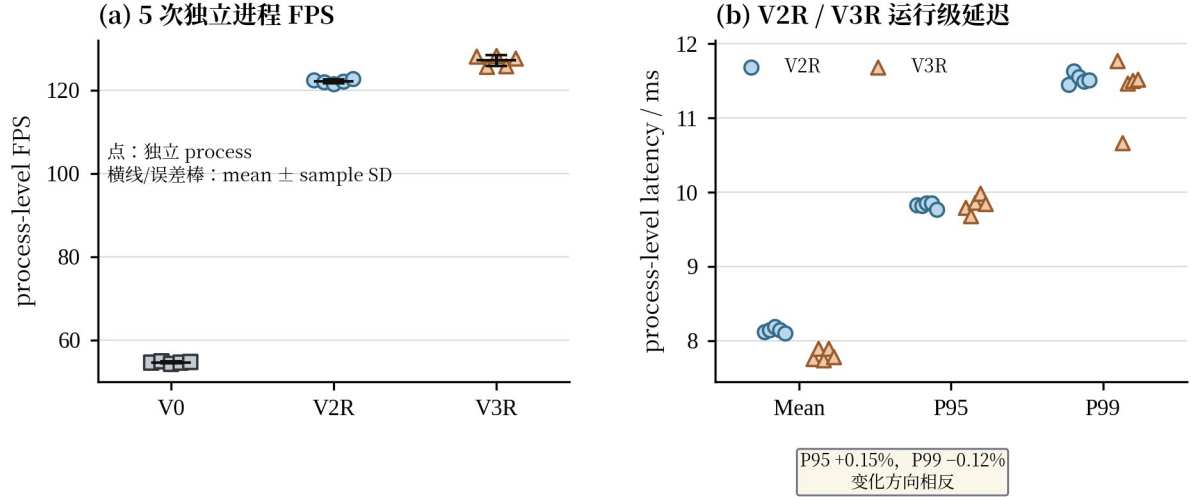


图4 运行级分布与尾延迟。各点为独立进程级描述量，横向偏移仅用于区分，不表示运行配对。

V3R 相对 V2R 的合并样本 P95 变化为 +0.15%，P99 变化为 -0.12%；两项变化均低于 0.2%且方向相反，未形成一致的尾延迟改善证据。平均性能和尾部结果应分别解读。

4.5 相关工作定位

表 4 的五维定位分别判断预处理是否移至 GPU、host 暂存是否成为变量、完整 E2E 是否覆盖预处理—模型执行—后处理/结果处理，以及路径比较是否验证任务正确性和报告均值外的百分位尾延迟。Kim et al.与 Tang & Qian 关注完整

边缘/Jetson 路径^[14,19]，PRESTO 关注 GPU 预处理^[8]，Shin & Kim 报告 Jetson 任务正确性^[21]，Bateni et al.研究 host/managed/pinned 内存但排除 CPU 预处理^[16]。本文据此定性定位固定检测器/Engine 下的路径证据，不作首次性、唯一性或跨论文性能排名。

分类见表 4。“完整 E2E”表示所报告边界覆盖预处理、模型执行及后处理/结果处理；未报告信息不视为否定，表中“明确否”仅用于原文明示排除的情形。

表 4 相关工作的定性定位。“未报告”不等同于“否”。

工作	GPU 预处理	host 内存策略	完整 E2E	任务正确性	尾延迟
Kim et al. (2025)	未报告	未报告	是	未报告	未报告
PRESTO (2025)	是	未报告	未报告	未报告	未报告
Tang & Qian (2024)	明确否	未报告	是	是	未报告
Shin & Kim (2022)	未报告	未报告	未报告	是	未报告
Bateni et al. (2020)	明确否	是	明确否	未报告	未报告
本文	是	是	是	是	是

4.6 局限性

本文仅评价单一 Jetson 平台、YOLOv8n TensorRT INT8 混合精度 Engine、NEU-DET 数据集和固定离线回放工作负载，未覆盖真实相机或跨平台/跨模型泛化；运行状态仅有非连续温度观察，没有连续遥测、功耗测量或独立归档的频率证据；实验也未测量阶段级 H2D/预处理时间及实际总线或 DRAM 流量，因而 40.96×名义载荷不能分配为单个组件的因果贡献。5 次独立进程只支持描述性结果，不支持统计显著性推断。

5 结论

本文重构了 Jetson 端 YOLOv8n TensorRT INT8 混合精度部署的输入形成与主机—设备数据路径。在冻结 180 幅测试图像上保持三条路径

任务级评价一致时，V2R 相对 V0 达到 2.24×FPS、平均延迟降低 55.45%，表明完整输入形成路径重构成受测配置中的主要 E2E 平均性能增量。

V3R 相对 V2R 的 FPS 变化为 +4.07%、平均延迟变化为 -4.03%，只体现固定实现中的有限平均增量；P95 为 +0.15%、P99 为 -0.12%，两项均低于 0.2%且方向相反，未形成一致的尾延迟改善证据。结论限于本文单一平台、检测器/Engine、数据集和离线回放工作负载，不作组件级因果或普适收益推断。

参考文献

- [1] Lv X, Duan F, Jiang J, 等. Deep Metallic Surface Defect Detection: The New Benchmark and Detection Network[J]. Sensors, 2020, 20 (6) : 1562. DOI:10.3390/s20061562.

- [2] Song K, Yan Y. A noise robust method based on completed local binary patterns for hot-rolled steel strip surface defects[J]. *Applied Surface Science*, 2013, 285: 858-864. DOI:10.1016/j.apsusc.2013.09.002.
- [3] Shao R, Zhou M, Li M, 等. TD-Net: tiny defect detection network for industrial products[J]. *Complex & Intelligent Systems*, 2024, 10: 3943-3954. DOI:10.1007/s40747-024-01362-x.
- [4] Chu Y, Yu X, Rong X. A Lightweight Strip Steel Surface Defect Detection Network Based on Improved YOLOv8[J]. *Sensors*, 2024, 24 (19) : 6495. DOI:10.3390/s24196495.
- [5] Zhang T, Pang H, Jiang C. GDM-YOLO: A Model for Steel Surface Defect Detection Based on YOLOv8s[J]. *IEEE Access*, 2024, 12: 148817-148825. DOI:10.1109/ACCESS.2024.3476908.
- [6] Ultralytics. Explore Ultralytics YOLOv8[EB/OL]. (2023) [2026-08-07]. <https://docs.ultralytics.com/models/yolov8/>.
- [7] Stäcker L, Fei J, Heidenreich P, 等. Deployment of Deep Neural Networks for Object Detection on Edge AI Devices With Runtime Optimization[C]//2021 IEEE/CVF International Conference on Computer Vision Workshops (ICCVW). IEEE, 2021: 1015-1022. DOI:10.1109/ICCVW54120.2021.00118.
- [8] Lee J, Han D, Kim J. PRESTO: Hybrid CPU-GPU Preprocessing Framework for Video-based AI Inference System[C]//Proceedings of the 23rd Annual International Conference on Mobile Systems, Applications and Services. ACM, 2025: 735-740. DOI:10.1145/3711875.3736688.
- [9] Weiss E, Caplan S, Horn K, 等. Real-Time Defect Detection in Electronic Components during Assembly through Deep Learning[J]. *Electronics*, 2024, 13 (8) : 1551. DOI:10.3390/electronics13081551.
- [10] Jacob B, Kligys S, Chen B, 等. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2018: 2704-2713.
- [11] Nagel M, Amjad R A, Van Baalen M, 等. Up or Down? Adaptive Rounding for Post-Training Quantization[C]//Proceedings of the 37th International Conference on Machine Learning: 卷 119. PMLR, 2020: 7197-7206.
- [12] NVIDIA Corporation. TensorRT 10.3 Release Notes[EB/OL]. (2024) [2026-08-07]. <https://docs.nvidia.com/deeplearning/tensorrt/archives/tensorrt-1030/pdf/TensorRT-Release-Notes.pdf>.
- [13] Kim N J, Lee J, Kim H. HyQ: Hardware-Friendly Post-Training Quantization for CNN-Transformer Hybrid Networks[C]//Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI-24. International Joint Conferences on Artificial Intelligence Organization, 2024: 4291-4299. DOI:10.24963/ijcai.2024/474.
- [14] Tang Y, Qian Y. High-speed railway track components inspection framework based on YOLOv8 with high-performance model deployment[J]. *High-speed Railway*, 2024, 2: 42-50. DOI:10.1016/j.hspr.2024.02.001.
- [15] NVIDIA Corporation. CUDA C++ Best Practices Guide: Release 12.6[EB/OL]. (2024) [2026-08-07]. <https://docs.nvidia.com/cuda/archive/12.6.0/cuda-c-best-practices-guide/index.html>.
- [16] Bateni S, Wang Z, Zhu Y, 等. Co-Optimizing Performance and Memory Footprint Via Integrated CPU/GPU Memory Management, an Implementation on Autonomous Driving Platform[C]//2020 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS). IEEE, 2020: 310-323. DOI:10.1109/RTAS48715.2020.00007.
- [17] Rodriguez M, Yarza I, Kosmidis L, 等. Analysis of GPU Memory Allocation Characteristics[C]//16th Workshop on Parallel Programming and Run-Time Management Techniques for Many-Core Architectures and 14th Workshop on Design Tools and Architectures for Multicore Embedded Computing Platforms (PARMA-DITAM 2025): 卷 127. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025: 1:1-1:15. DOI:10.4230/OASlcs.PARMA-DITAM.2025.1.
- [18] NVIDIA Corporation. CUDA C++ Programming Guide: Release 12.6[EB/OL]. (2024) [2026-08-07]. <https://docs.nvidia.com/cuda/archive/12.6.0/cuda-c-programming-guide/index.html>.
- [19] Kim S, Kim C, Kim S. Improving Performance of Real-Time Object Detection in Edge Device Through Concurrent Multi-Frame Processing[J]. *IEEE Access*, 2025, 13: 1522-1533. DOI:10.1109/ACCESS.2024.3520240.
- [20] Dean J, Barroso L A. The Tail at Scale[J]. *Communications of the ACM*, 2013, 56 (2) : 74-80. DOI:10.1145/2408776.2408794.
- [21] Shin D-J, Kim J-J. A Deep Learning Framework Performance Evaluation to Use YOLO in Nvidia Jetson Platform[J]. *Applied Sciences*, 2022, 12 (8) : 3734. DOI:10.3390/app12083734.
- [22] Lema D G, Sánchez-González L, Usamentiaga R, 等. Benchmarking deep learning models for surface defect detection: a reproducible and statistically-rigorous approach[J/OL]. *Journal of Intelligent Manufacturing*, 2025. DOI:10.1007/s10845-025-02672-8.